7. אופרטור התמיכה המבנית

זו הליבה המתמטית של המתכנן. הפונקציה `src/layerforge_l1/reveal_planner.py:171` עונה על השאלה: **אם מותר לי להשתמש בחומר רק בתוך הקבוצה U, איזה חלק ממנו אפשר להשאיר כך שהוא מחובר למסגרת דרך מעברים ברוחב 1.27 מ"מ לפחות?**

7.1 הגדרות: שחיקה, התפשטות ועיגון

THEORETICAL BACKGROUND מורפולוגיה בינארית. A היא קבוצת פיקסלים ו־K גרעין סימטרי שמרכזו בראשית:

$$A \ominus K = \{p:\ p + K \subseteq A\}, \qquad A \oplus K = \{p + k:\ p \in A,\ k \in K\}, \qquad A \circ K = (A \ominus K) \oplus K$$

הפתיחה $A \circ K$ היא איחוד כל ההזזות של K שנמצאות כולן בתוך A, כלומר כל המקומות ש"מטבע" בצורת K יכול לכסות בלי לצאת מ־A.

IMPLEMENTED MATHEMATICS `cv2.erode` מחשב $\min_{k \in K} A(p+k)$ ו־`cv2.dilate` מחשב $\max$, והם ממשים בדיוק את ההגדרות האלה. **פרט חשוב** (הודגם בדוגמה A2 בהמשך): בשחיקה OpenCV מתייחס לפיקסלים **מחוץ לתמונה** כאל חומר (ערך גבול מקסימלי). ברשת האמיתית התמונה מוקפת במסגרת ברוחב 128 פיקסלים, שרחבה בהרבה מהגרעין, ולכן לזה אין השפעה על התוצאה.

IMPLEMENTED MATHEMATICS **עיגון:** `src/layerforge_l1/reveal_planner.py:216`

$$\mathrm{Anch}_P(C) = \bigcup \{Q:\ Q \text{ is a 4-connected component of } C,\ Q \cap P \neq \emptyset\}$$

הקוד מחשב את הרכיבים ב־`cv2.connectedComponents(connectivity=4)`, מסמן את התוויות שמופיעות ב־$C \cap P$, ומחזיר מסכה של כל הפיקסלים שלהן.

src/layerforge_l1/reveal_planner.py:211-234

```python
def _orphan_cores(material: np.ndarray, perimeter: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    core = cv2.erode(material.astype(np.uint8), kernel).astype(bool)
    return core & ~_anchored(core, perimeter)


def _anchored(core: np.ndarray, perimeter: np.ndarray) -> np.ndarray:
    count, components = cv2.connectedComponents(core.astype(np.uint8), connectivity=4)
    keep = np.zeros(count, dtype=bool)
    keep[np.unique(components[perimeter & core])] = True
    keep[0] = False
    return keep[components]


_CROSS = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3))


def _geodesic_reach(seed: np.ndarray, domain: np.ndarray, steps: int) -> np.ndarray:
    reach = seed & domain
    for _ in range(steps):
        grown = cv2.dilate(reach.astype(np.uint8), _CROSS).astype(bool) & domain
        if np.array_equal(grown, reach):
            break
        reach = grown
    return reach
```

7.2 האופרטור הבסיסי

src/layerforge_l1/reveal_planner.py:183-185

```python
    covered_core = cv2.erode(covered.astype(np.uint8), kernel).astype(bool)
    anchored = _anchored(covered_core, perimeter)
    support = cv2.dilate(anchored.astype(np.uint8), kernel).astype(bool) & covered
```

IMPLEMENTED MATHEMATICS

$$\mathrm{Struct}_0(U) = \left(\mathrm{Anch}_P\left(U \ominus K\right) \oplus K\right) \cap U$$

שלב 1: $U \ominus K$ היא ה"ליבה" (core): כל הפיקסלים שאפשר למקם עליהם את מרכז ה"מטבע" ברוחב 1.27 מ"מ. **שלב 2:** Anch_P משאיר רק חלקי ליבה שנוגעים במסגרת. **שלב 3:** $\oplus K$ מחזיר את הרוחב: כל פיקסל ש"מטבע" כזה, שמחובר למסגרת, מכסה. **שלב 4:** $\cap U$ הוא ביטוח: לא יוצרים חומר מחוץ ל־U.

DERIVED **פירוש:** $\mathrm{Struct}_0(U)$ הוא החלק של הפתיחה $U \circ K$ שמטבע ברוחב 1.27 מ"מ מגיע אליו **בהחלקה רציפה מהמסגרת**, בלי לצאת מ־U. שני פיקסלי ליבה סמוכים (4 שכנים) p ו־q מקיימים $p + K \subseteq U$ ו־$q + K \subseteq U$, ולכן מסלול ליבה 4־קשיר מתאר "מסדרון" ברוחב הגרעין לכל אורכו. זה בדיוק מה שה־validator בודק (פרק 10).

DERIVED **אינווריאנט:** $\mathrm{Struct}(U) \subseteq U$. בשורות 185 ו־207 יש `& covered` מפורש. בשורה 196 (גשרים) אין, אבל `bridge` $\subseteq$ `covered_core` $= U \ominus K$. מההגדרה, לכל $b \in U \ominus K$ מתקיים $b + K \subseteq U$, ולכן bridge $\oplus K \subseteq U$ (K סימטרי). **זה האינווריאנט שמבטיח שהמתכנן לא מוסיף חומר במקומות שצריכים להישאר חורים** (פרק 9).

7.3 מונוטוניות: הבסיס לכל המתכנן

DERIVED **טענה:** אם $U \subseteq U'$, אז $\text{Struct}_0(U) \subseteq \text{Struct}_0(U')$.
נימוק: (1) שחיקה מונוטונית: $U \ominus K \subseteq U' \ominus K$. (2) כל רכיב Q של הליבה הקטנה שנוגע ב־P מוכל ברכיב Q' של הליבה הגדולה (הוא קשיר וכלול בה), ו־Q' נוגע ב־P באותו פיקסל. לכן Anch מונוטוני. (3) גם התפשטות וחיתוך מונוטוניים.
הסתייגות: לולאות התיקון (גשרים וחיתוכים, בהמשך) הן היוריסטיות, והנימוק הזה לא מכסה אותן. ה־docstring של הקובץ טוען למונוטוניות, אבל לגבי הגרסה המלאה עם התיקונים זו הנחה עבודה, לא משפט מוכח.

המשמעות המעשית: **כשקבוצת המכוסים גדלה, התמיכה רק גדלה.** מכאן נובע שחשיפה של יותר תאים מוקדם לא מזיקה לאף לוח מאחור (פרק 8).

7.4 לולאת הגשרים (bridge loop)

```
src/layerforge_l1/reveal_planner.py:186-196
186      for _ in range(8):
187          core = cv2.erode(support.astype(np.uint8), kernel).astype(bool)
188          orphan = core & ~_anchored(core, perimeter)
189          if not orphan.any():
190              break
191          bridge = (
192              cv2.dilate(orphan.astype(np.uint8), _CROSS).astype(bool) & covered_core & ~core
193          )
194          if not bridge.any():
195              break
196          support |= cv2.dilate(bridge.astype(np.uint8), kernel).astype(bool)
```

הבעיה שהיא פותרת: ברסטר, $(\cdot \oplus K) \ominus K$ אינו בהכרח שווה לקלט. אחרי ההתפשטות, שחיקה חוזרת (בדיוק מה שה־validator עושה) עלולה ליצור פיקסלי ליבה "תועים" שנוגעים בליבה המעוגנת **רק באלכסון**. ה־validator עובד בקשירות 4, ולכן יספור אותם כ"ליבות יתומות" (orphan cores) ויכשיל את הלוח. זו הייתה התקלה של "מדרגות אלכסוניות" שתוקנה במהלך הפיתוח.

IMPLEMENTED MATHEMATICS בכל איטרציה (עד 8):

$$O = (S \ominus K) \setminus \text{Anch}_P(S \ominus K), \qquad B = (O \oplus C^{+}_{3\times 3}) \cap (U \ominus K) \setminus (S \ominus K), \qquad S \leftarrow S \cup (B \oplus K)$$

C^{+} הוא צלב 3×3, כלומר 4 שכנים. B הם פיקסלים שצמודים (4 שכנים) ליתום, נמצאים בליבה המותרת של U, אבל לא נמצאים כרגע בליבה של S. הוספת $B \oplus K$ ל־S הופכת אותם לליבה ומחברת את היתום בחיבור 4־קשיר.

7.5 לולאת החיתוך (growing-cut prune)

```
src/layerforge_l1/reveal_planner.py:197-208
197      # Where no bridge fits, cut the anchored core next to each orphan so the
198      # material it implied is dropped (hidden) or reported unsupported (visible).
199      for attempt in range(16):
200          core = cv2.erode(support.astype(np.uint8), kernel).astype(bool)
201          orphan = core & ~_anchored(core, perimeter)
202          if not orphan.any():
203              break
204          size = 5 + 4 * attempt  # grow the cut if the same orphan persists
205          cut = cv2.dilate(orphan.astype(np.uint8), np.ones((size, size), np.uint8)).astype(bool)
206          anchored = _anchored(anchored & ~cut, perimeter)
207          support = cv2.dilate(anchored.astype(np.uint8), kernel).astype(bool) & covered
208      return support
```

IMPLEMENTED MATHEMATICS אם נשארו יתומים, ניסיון t (עד 16):

$$c_t = 5 + 4t, \quad X = O \oplus \square_{c_t}, \quad A \leftarrow \mathrm{Anch}_P(A \setminus X), \quad S \leftarrow (A \oplus K) \cap U$$

$\square_c$ הוא ריבוע מלא c×c (5, 9, 13 וכן הלאה). חותכים את הליבה המעוגנת סביב היתום, ואם היתום נשאר, החיתוך גדל. החומר שהיתום "ייצג" נעלם מהתמיכה: אם הוא היה מוסתר, זה בטוח, ואם הוא היה גלוי, `unsupported_pixels` יחזיר אותו כפיקסל לא נתמך (פרק 8).

LIMITATION אחרי 8 גשרים ו־16 חיתוכים הפונקציה מחזירה את S גם אם נשארו בו יתומים. במקרה כזה ההגנה היא בשכבה העליונה: `unsupported_pixels` בודק את הלוח הגמור (פרק 8.2), וה־validator בודק כל לוח בסוף.

7.6 הגעה גאודזית: הגבול ל"בליטות" (nubs)

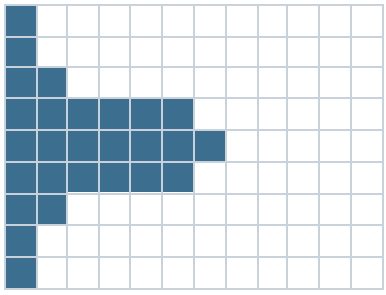
src/layerforge_l1/reveal_planner.py:227 IMPLEMENTED MATHEMATICS

$$R_0 = S \cap D, \qquad R_{t+1} = (R_t \oplus C^+) \cap D, \qquad D = S \cup V, \quad t \le 5$$

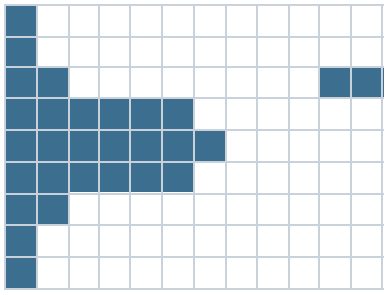
התוצאה: כל פיקסל גלוי שאפשר להגיע אליו מהתמיכה S במסלול 4־קשיר באורך 5 צעדים לכל היותר, שעובר רק דרך פיקסלים של $S \cup V$. אלו קצוות של קווים ופינות חדות שבולטים עד 0.49 מ"מ מעבר לחומר הנושא.

7.7 דוגמה A2: הפונקציה האמיתית על רשת 12×9

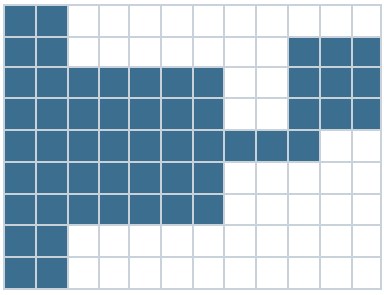
בדוגמה הזו קראתי ל־`structural_support` ול־`_anchored` **מתוך הקוד**, עם גרעין אליפטי 3×3 (שהוא צלב) במקום 13×13, כדי שהמצבים יהיו קריאים. המסגרת P היא העמודה השמאלית. U כולל גוף ברוחב 5 שמחובר למסגרת, "זיז" ברוחב פיקסל אחד, ובלוק 3×3 שמחובר לגוף רק דרך הזיז.



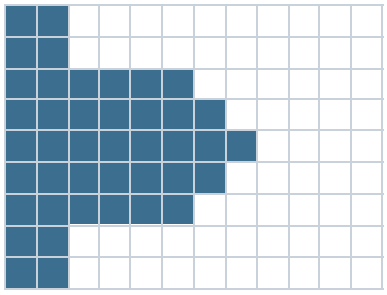
$\mathrm{Anch}_P(U \ominus K)$



$U \ominus K$



U (covered)



`structural_support`

- **ליבה:** הזיז נעלם בשחיקה כי הוא צר מהגרעין. הבלוק שבימין משאיר ליבה (עמודות 10–11 בשורה 2). את עמודה 11 משאירה רק התנהגות הגבול של OpenCV, שמתייחס לפיקסלים מחוץ לתמונה כחומר. בשורות 0–1 ו־7–8 ליד עמודה 0 קורה אותו דבר.
- **עיגון:** ליבת הבלוק לא מחוברת למסגרת, ולכן היא נזרקת. היא "יתום".
- **תמיכה:** הגוף חוזר לרוחבו המלא ($\oplus K$), ובנוסף פיקסל אחד מהזיז (שורה 4, עמודה 7), שנמצא בטווח הגרעין מליבת הגוף. הבלוק והחלק החיצוני של הזיז **אינם** בתמיכה.
- **מה זה אומר למתכנן:** נניח שהבלוק אמור להיות גלוי בלוח הזה. אם הזיז אינו גלוי, ההגעה הגאודזית עוברת רק דרך $S \cup V$ ולא מגיעה לבלוק, ולכן הבלוק נדחה מיד. אם גם הזיז גלוי, הבלוק נמצא 2–3 צעדים מהתמיכה ולכן "מושג". אבל אז בדיקת הלוח הגמור (8.2) מוצאת ליבה יתומה בתוך הבלוק, והוא נדחה בכל זאת.

7.8 סיבוכיות

DERIVED לרשת $N \times N$ ($N = 2304$, כ־5.3 מיליון פיקסלים) וגרעין עם $|K| = 121$: כל שחיקה או התפשטות עולה $O(N^2|K|)$ במקרה הגרוע. OpenCV ממומש ב־SIMD, ולכן הזמן בפועל קטן בהרבה מהחסם. רכיבים קשירים עולים $O(N^2)$. קריאה אחת כוללת לפחות 2 שחיקות, התפשטות ותיוג, ועוד עד 8 ו־16 סבבי תיקון. בפועל רוב הקריאות יוצאות כבר בסבב הראשון.